

从访存流视角分析 MoE 推理：面向片上加速器的 Bank 感知虚拟化与预取

架构名称： MemDomain

论文定位： 本文是在本科毕业设计基础上的完善与延伸。相比本科论文主要验证统一总线、动态 Bank 分配和预取机制的可行性，DATE 论文进一步从 **动态 Bank 级访存流** 出发，系统分析 **静态 Bank 所有权失配** 与 **计算-预取冲突**，并据此提出面向 MoE 的片上存储协同优化架构。

目标篇幅： DATE 正文最多 6 页。

全文主线：

MoE 路由与专家异构

→ IA / Weight / OA 访存流随时间变化

→ 静态 Bank 划分失配与预取干扰

→ 统一存储域 + 动态 Bank 虚拟化 + Bank 感知 Chunk 预取。

摘要

混合专家模型（Mixture-of-Experts, MoE）通过稀疏专家激活扩展模型容量。然而，token-to-expert 路由使不同专家在不同执行阶段接收不同数量的 Token，同时异构专家还可能具有不同的参数规模与计算规模。这些因素共同导致输入激活（IA）、权重（Weight）和输出激活（OA）的片上访存需求随专家、层和计算 Tile 动态变化。

传统加速器通常采用固定 SRAM 区域与静态 Bank 绑定。本文的性能刻画表明，MoE 的瓶颈不仅来自总容量或总带宽不足，更来自动态 Bank 级访存流与固定资源所有权之间的失配。该失配会同时造成 Bank hotspot 与 Bank idle，而传统预取还可能与正在执行的计算请求争用 Bank 容量、端口和带宽。

为此，本文提出 **MemDomain**，一种面向 MoE 加速器的统一虚拟化片上存储域。MemDomain 将普通 SRAM 访问与累加写回统一到同一套 Bank 管理路径下，根据运行时压力动态建立虚拟 Bank 到物理 Bank 的映射，并将 Chunk 级预取与当前计算访存流量协同调度。与专家级预测方法不同，MemDomain 在专家选择完成后工作，重点决定未来专家权重 Chunk 应在何时、以何种粒度、放入哪个片上 Bank。

与静态划分和传统预取相比，MemDomain 将 Bank 冲突降低 **[X%]**，将访存停顿降低 **[Y%]**，将总执行周期降低 **[Z%]**，对应的面积和功耗开销分别为 **[A%]** 和 **[P%]**。

关键词： 混合专家模型，片上存储，Bank 虚拟化，访存流，预取

I. 引言

A. 研究背景与问题

- MoE 已成为扩展大语言模型参数规模的重要架构。
- 稀疏路由虽然减少了单个 Token 激活的专家数量，但也将原本规则的稠密 FFN 执行转化为运行时依赖的动态专家执行。
- 每个被路由的专家都会产生：
 - IA 读取；
 - 专家权重读取；
 - OA / Accumulator 写回。

- 专家 Token 数、专家规模、top-k 设置以及计算 Tile 阶段共同决定三类数据流量。
- 因此，MoE 推理不应只被笼统描述为“Memory-Bound”，而应进一步被视为一个 **动态 Bank 级访存流问题**。

建议核心句：

稀疏专家激活降低了算术计算量，却引入了随专家、层和计算 Tile 变化的 IA、Weight 与 OA 动态访存流。

B. 现有工作为什么不足

1. 路由与负载均衡

- 现有工作主要优化 Token 数量、路由概率或专家执行负载。
- Token 数量均衡并不等价于访存流量均衡：
 - 专家规模可能不同；
 - IA、Weight 和 OA 的位宽与生命周期不同；
 - top-k 复制会非均匀增加访存请求。
- 此类工作并不直接管理路由完成后的片上 Bank 竞争。

2. 通信、卸载以及 STEP 类专家预取

- 现有系统主要优化：
 - 专家放置；
 - 跨 GPU All-to-All；
 - CPU-GPU 专家卸载；
 - 专家缓存与专家预测。
- STEP 主要决定 **哪些专家应该被提前加载或保留**。
- MemDomain 则决定 **已被选中专家的数据 Chunk 应何时进入片上 SRAM，以及放到哪个 Bank**。
- 两者关注的层级不同，可以相互补充。

建议核心句：

专家级预取回答“未来需要哪个专家”，而 MemDomain 回答“该专家的数据应何时、以何种粒度、放入哪个片上 Bank”。

3. 传统片上存储优化

传统加速器通常假设：

- Tensor 分区固定；
- Tensor 生命周期规则；
- Scratchpad 与 Accumulator 资源归属固定；
- 数据搬入前即可确定目标区域。

这些假设难以适应路由相关的 MoE 动态访存流。

C. 关键观察

观察 1: Bank 需求随时间变化

不同专家的 Token 数和参数规模差异，使 IA、Weight 和 OA 的 Bank 需求随执行阶段显著变化。

观察 2: 不存在全局最优的静态 Bank 比例

某一专家或 Tile 下合理的 Bank 配置，可能在下一阶段产生 hotspot 和 idle Bank。

观察 3: 预取可能放大 Bank 冲突

预取可以隐藏数据搬运延迟，但也会提前占用 Bank 容量和端口。若忽略当前计算流量，预取可能增加而不是减少 Stall。

D. 方案概述

MemDomain 使用三个机制解决上述问题：

1. 统一片上存储域

- 将 SRAM 普通读写和 Accumulator 写回暴露给统一分配器；
- 使原本割裂的片上资源可以被共同管理。

2. 动态虚拟 Bank 到物理 Bank 映射

- 将软件可见的逻辑数据对象与物理 Bank 解耦；
- 在 Chunk 生命周期边界，根据运行时压力分配物理 Bank。

3. Bank 感知 Chunk 预取

- 联合考虑预取距离、Chunk 粒度、当前 Bank 压力和目标 Bank；
- 在隐藏搬运延迟的同时，避免与当前计算流量发生冲突。

E. 主要贡献

1. Bank 级访存流刻画与建模

- 建立专家激活、专家规模、IA/Weight/OA 请求、Bank hotspot、Bank imbalance 与 memory stall 之间的联系。
- 在升级后的 SCALE-Sim 中加入 MoE、Bank 冲突、动态分配、Chunk 预取和多 GPU/EP 抽象。

2. MemDomain 架构

- 将普通 SRAM 访问与累加访问统一到同一 Bank 管理路径。
- 通过虚拟 Bank 到物理 Bank 的动态映射实现运行时资源分配。

3. Bank 感知 Chunk 预取

- 协同优化预取窗口、Chunk 粒度、目标 Bank、资源占用和计算访存。
 - 在不同 top-k、专家数、路由倾斜、Token 数以及 EP 配置下评估鲁棒性。
-

II. 研究动机与性能瓶颈分析

本章目的：

先确定性能瓶颈在哪里，再分析为什么出现该瓶颈，最后提炼三个设计 Challenge。

A. MoE 执行与片上访存流

1. MoE 执行流程

- Router 将 Token 分发给 top-k 专家。
- 在 EP 模式下，不同专家分布在不同 GPU。
- 每个专家执行两次 FFN GEMM：
 - FF1；
 - FF2。
- 每一阶段均产生：
 - IA 读取；
 - Weight 读取；
 - OA / Accumulator 写回。

2. Bank 化片上存储模型

- SRAM 由多个物理 Bank 组成。
- 每个 Bank 具有有限的：
 - 容量；
 - 带宽；
 - 端口数。
- 同一周期或同一时间窗口内访问同一 Bank 的多个请求需要排队，并最终产生 Stall。

对于专家 (e) 在执行阶段 (t) 的片上访存流，可定义：

$$\begin{aligned} &[\\ &\mathbf{F}_e(t) = \\ &\left[\right. \\ &F_e^{IA}(t), \\ &F_e^W(t), \\ &F_e^{OA}(t) \\ &\left. \right] \end{aligned}$$

其中：

- $(F_e^{IA}(t))$ 主要受分配给专家 (e) 的 Token 数影响；
- $(F_e^W(t))$ 主要受专家规模和当前 Weight Chunk 影响；
- $(F_e^{OA}(t))$ 主要受 Token 数、输出维度和累加位宽影响。

专家 (e) 的总访存需求可以写为：

[
 $D_e(t) = D_e^{\{IA\}}(t) + D_e^{\{W\}}(t) + D_e^{\{OA\}}(t)$
]

但三类需求并不会同比例变化，因此固定 Bank 比例不能持续匹配不同阶段。

3. 研究范围

- 专家采用 EP 模式进行映射。
- 默认配置：
 - 2 张 GPU；
 - 每张 GPU 4 个专家；
 - top-k 可配置，默认根据实验选择 top-1 或 top-2。
- 当前 GPU / Core 进行详细仿真。
- 其他 GPU / Core 以抽象 expert workload 的形式产生访存压力。
- 跨 GPU 通信作为系统约束建模，但不是本文主要优化对象。

B. 性能瓶颈刻画

研究问题

系统的主要性能瓶颈是否集中在 MoE 专家执行中？该瓶颈主要来自片上访存服务，还是计算吞吐不足？

实验 B1：MoE 与非 MoE 层周期分解

对比对象

- Attention Q/K/V 投影；
- Attention Score / Attention Output；
- Router；
- Expert FF1；
- Expert FF2。

采集指标

- Total Cycles；
- Compute Cycles；
- Memory Stall Cycles；
- Stall Ratio；
- Bank Conflict Count / Rate；
- Bank Imbalance；
- IA / Weight / OA Stall Breakdown。

分析目标

- 比较 MoE 与非 MoE 层。
- 验证 MoE 层是否贡献了主要 Memory Stall。
- 验证高 Stall Ratio 是否伴随：
 - 更高的 Bank Imbalance;
 - 更高的 Bank Conflict Rate。
- 排除“性能下降仅仅由计算规模变大造成”的解释。

建议结论：

MoE 层的 Memory Stall 明显高于非 MoE 层，而纯计算周期的增加相对有限。其 Stall Ratio 与 Bank Imbalance、Bank Conflict Rate 呈明显相关性，说明不均衡的片上存储服务是主要瓶颈。

建议图

图 1：MoE 执行流程与性能瓶颈刻画

- (a) MoE Pipeline 与 IA/Weight/OA 数据流;
- (b) MoE 与非 MoE 层周期分解;
- (c) 各层 Stall Ratio 与 Bank Imbalance。

C. 根因分析与设计 Challenge

本节通过三个 Characterization 实验，分析 Bank Stall 的根本来源以及传统预取的局限。

C1. IA / Weight / OA 需求随时间变化

实验 C1

按专家、FF1/FF2 阶段和计算 Tile 记录：

- IA Traffic;
- Weight Traffic;
- OA Traffic;
- 理想或实际选择的 Bank 比例;
- 每阶段 Working Set;
- Active Bank Demand。

实验需要证明

- 不同专家接收的 Token 数不同;
- 不同专家的参数规模不同;
- IA、Weight 和 OA 的需求独立变化;
- 不同专家和不同执行阶段对应的最优 Bank 比例不同。

Challenge 1

Challenge 1: 如何在统一片上存储视角下暴露并管理异构、时变的 IA、Weight 与 OA 需求。

对应设计要求:

- 原本割裂的资源需要暴露给统一分配器;
 - SRAM 普通访问和 Accumulator 相关访问不能完全相互隔离。
-

C2. 静态 Bank 所有权失配

实验 C2

保持以下参数不变:

- 总 Bank 数;
- SRAM 总容量;
- 总带宽;
- 端口数;
- 工作负载;
- 计算阵列。

只改变静态 IA:Weight:OA Bank 比例。

示例配置:

- 8:8:8;
- 7:10:7;
- 4:14:6;
- 12:8:4;
- 2:11:11;
- 其他具有代表性的静态比例。

采集指标

- Total Cycles;
- Memory Stall Cycles;
- Bank Conflict Count;
- Hotspot Bank Ratio;
- Idle Bank Ratio;
- Per-bank Queue Length;
- Bank Utilization;
- Bank Imbalance;
- Per-stage Best Static Ratio。

实验需要证明

- 不同阶段对应的较优 Bank 比例不同；
- 对某一阶段有效的静态配置，可能对下一阶段无效；
- 即使总容量足够，也可能同时出现 hotspot 和 idle Bank；
- 问题不只是总资源不足。

建议结论：

性能瓶颈不仅来自总容量或总带宽不足，更来自时变访存流与固定 Bank 所有权之间的失配。

Challenge 2

Challenge 2: 静态 Bank 所有权无法跟随动态访存流，需要低开销的虚拟 Bank 到物理 Bank 重映射。

对应设计要求：

- 逻辑数据对象与物理 Bank 解耦；
- 数据加载前完成分配；
- Chunk 在其有效生命周期内保持映射稳定；
- 生命周期结束后释放；
- 尽量避免运行中数据迁移。

C3. 计算—预取 Bank 干扰

实验 C3

比较：

- No Prefetch；
- Naive Prefetch。

Naive Prefetch 定义：

- 仅根据未来使用距离触发；
- 不检查当前 Bank 压力；
- 写入静态目标区域或固定 Bank 集合。

Sweep：

- Prefetch Window ($W = 0, 1, 2, 4, 8$)；
- Chunk Size = 1、2、4、8 Tiles；
- 路由倾斜程度；
- 专家规模。

Bank (b) 的总请求量定义为：

[
 $R_b^{\text{total}}(t) = R_b^{\text{compute}}(t) + R_b^{\text{prefetch}}(t)$
]

采集指标

- Prefetch Requests;
- Prefetch Traffic;
- Compute–Transfer Overlap;
- Prefetch-induced Bank Conflicts;
- Compute Stall Cycles;
- Late Prefetch Ratio;
- Timely Prefetch Ratio;
- Unused Prefetch Ratio;
- Bank Occupancy;
- Total Cycles。

实验需要证明

- Window 太小，预取来不及；
- Window 太大，过早占用 Bank 并加剧冲突；
- Chunk 太小，控制和映射开销上升；
- Chunk 太大，放置灵活性下降；
- 即使预取数据最终被使用，只要干扰当前计算，也可能降低总体性能。

Challenge 3

Challenge 3：预取必须在传输及时性与计算—预取 Bank 干扰之间取得平衡，并联合考虑窗口、Chunk 粒度和目标 Bank。

对应设计要求：

- 预取时机和目标 Bank 必须协同决定；
- Bank 压力过高时，应延迟、重定向或取消预取。

D. Challenge 与机制对应关系

性能问题	Challenge	MemDomain 机制
IA/Weight/OA 需求随时间变化	C1	Unified On-Chip Memory Domain
静态 Bank 所有权失配	C2	Dynamic Bank Virtualization
计算与预取相互干扰	C3	Bank-Aware Chunk Prefetching

III. MemDomain 架构

A. 总体架构

1. 基线平台

简要介绍 Buckyball，只保留与本文相关的部分：

- Frontend;
- Midend;
- Backend;
- BallDomain;
- DRAM / DMA;
- Unified SRAM Banks。

2. 整体执行流程

1. Router / EP Mapping 确定专家执行顺序。
2. 专家计算被拆分为 Tile 和 Weight Chunk。
3. 当前 Tile 执行。
4. Prefetch Controller 根据窗口选择未来 Chunk。
5. Bank Allocator 检查容量、Bank 压力和映射状态。
6. 为虚拟 Chunk 分配一个或多个物理 Bank。
7. 当前计算执行时，预取数据并行搬入。
8. Chunk 被消费后，在生命周期结束时释放。

3. 建议架构图

图 2：MemDomain 总体架构

图中必须包含：

- Compute Requests;
 - DMA Requests;
 - Prefetch Requests;
 - 统一 BankRead / BankWrite;
 - Mapping Table;
 - Virtual Bank ID;
 - Physical Bank ID;
 - Bank Pressure Information;
 - Unified SRAM Banks;
 - 当前详细 GPU 与远端黑箱 GPU 压力。
-

B. 统一片上存储域

目标

解决资源割裂，并使所有可用 Bank 对统一分配器可见。

核心句：

统一访问路径并不只是接口简化，而是使所有片上 Bank 资源能够被统一分配器共同管理。

机制

- 普通读写与累加写回统一为 BankRead/BankWrite 语义。
- Midend 汇聚：
 - BallDomain 请求；
 - DMA 请求；
 - Prefetch 请求。
- AccPipe 完成：
 - `wmode = 0` 时直接写；
 - `wmode = 1` 时读一改一写累加。
- 原本不能互相借用的资源进入统一资源池。

论文定位

- 统一访问路径是动态虚拟化的硬件基础。
- 除非补充资源割裂实验，否则不要将它单独宣传成最主要性能根因。
- 真正的架构核心是统一路径之上的动态资源分配。

C. 动态 Bank 虚拟化

如果当前实现真实使用 Bank 压力或访问统计进行选择，可以使用“Memory-Flow-Aware Bank Virtualization”。

如果当前只选择第一个空闲 Bank，建议暂时使用“Dynamic Bank Virtualization”。

1. 映射对象和粒度

每个虚拟数据对象建议包含：

- `vbank_id`；
- Tensor Type；
- Expert ID；
- Chunk ID；
- Size；
- Lifetime；
- Access Mode；
- Group Count；

- Estimated / Measured Bank Pressure。

推荐映射粒度：

- Weight Chunk / Compute Tile;
- 不采用逐元素映射;
- 不采用专家级永久绑定。

2. 分配时机

- 数据加载前完成映射分配;
- 使用类似 `mset` 的操作建立映射;
- `mvin` 根据已有映射完成数据搬入;
- Chunk 在有效生命周期内保持绑定;
- 生命周期结束后释放;
- 不需要运行中原地迁移。

建议核心句：

MemDomain 采用分配时重映射，而不是运行时原地迁移。一个 Chunk 在搬入前完成分配，并在整个有效生命周期内保持绑定。

3. Bank 选择策略

至少需要定义一个可实现的选择规则。

示例：

```
[  
b^*=\arg\min_b  
\left(  
\alpha Q_b+\beta U_b+\gamma I_b  
\right)  
]
```

其中：

- (Q_b): 当前请求队列压力;
- (U_b): 近期或预测利用率;
- (I_b): 与当前计算流量的冲突风险。

如果当前实现仅选择第一个空闲 Bank：

- 要么增加 Bank Pressure Counter;
- 要么将表述改为动态空闲 Bank 分配，而不是 memory-flow-aware。

4. Multi-Bank Group

需要说明：

- 一个虚拟 Chunk 如何映射到多个物理 Bank;
- `group_id` 如何选择物理 Bank;
- 连续 Beat / Chunk 如何条带化分布;

- 如何提高有效并行度；
- `group_count` 如何确定。

5. 映射开销

必须说明：

- Mapping Table 字段；
- 查询延迟；
- 分配延迟；
- 释放延迟；
- 表项数量；
- 查询是否流水化；
- 映射操作能否与计算或搬运重叠。

D. Bank 感知 Chunk 预取

1. 与专家预测的区别

建议开头直接写：

与专家级预测方法不同，MemDomain 在专家选择完成后工作。给定专家执行顺序后，MemDomain 以 Tile 为粒度预取未来 Weight Chunk，并决定其何时、放入哪个片上 Bank。

MemDomain 不负责预测哪个专家被激活，而是消费：

- Routed Expert Order；
- Expert Workload；
- Tile / Chunk Schedule。

2. 预取定义

- **Prefetch Window (W)**：当前计算 Tile 与被预取 Weight Chunk 的距离。
- **Chunk Size**：一次预取搬运的 Tile 数或字节数。
- **Initial Chunk**：1 Tile。
- 允许计算与预取部分重叠。
- 主要预取对象：
 - Expert Weight；
 - IA 仅在架构明确支持时加入。
- OA 不进行预取。

3. 预取状态

建议定义：

- `NotIssued`；
- `InFlight`；

- Ready ;
- Consumed ;
- Delayed ;
- Cancelled ;
- Unused / Evicted 。

4. 预取决策流程

预取发起前：

1. 检查 Chunk 是否已驻留；
2. 估计 Time-to-Use；
3. 检查空闲容量；
4. 检查候选 Bank 压力；
5. 估计可重叠时间；
6. 决定：
 - 立即发起；
 - 延迟；
 - 重定向到其他 Bank / Group；
 - 取消。

5. Bank 感知放置

Naive Prefetch：

- 只看未来距离；
- 写入固定 Bank 或静态区域。

Bank-Aware Prefetch：

- 同时考虑：
 - Time-to-Use；
 - Queue Pressure；
 - Bank Occupancy；
 - 与当前 Compute 的冲突；
 - Multi-Bank Group 可用性。

6. 执行时间线

图 3：不同方案的执行时间线

建议展示四种配置：

1. Static + No Prefetch；
2. Static + Naive Prefetch；
3. Dynamic + No Prefetch；
4. Full MemDomain。

该图需要说明：

- Transfer-Compute Overlap;
- Naive Prefetch 下的 Bank 冲突;
- Bank-Aware Placement 如何降低冲突;
- Mapping 与 Prefetch 如何重叠。

E. EP 集成

- 专家在 EP 模式下——映射。
- 每张 GPU 独立管理本地 MemDomain。
- 当前 GPU 详细仿真。
- 其他 GPU 产生抽象 Expert Workload 和访存压力。
- 跨 GPU 通信作为时序约束。
- MemDomain 不修改：
 - Token Dispatch 语义;
 - All-to-All 正确性;
 - Expert Selection 语义。
- 若没有实现通信优化，不宣称优化 All-to-All。

建议核心句：

■ 本文将跨 GPU 通信建模为系统级约束，重点研究其与本地片上访存执行之间的相互影响。

IV. 实验评估

A. 实验设置

1. 平台

- 升级后的 SCALE-Sim：
 - MoE Workload;
 - Bank Conflict Model;
 - Dynamic Bank Allocation;
 - Chunk-level Prefetch;
 - Multi-GPU / EP Abstraction。
- Buckyball RTL：
 - 功能验证;
 - 映射正确性;
 - 统一访问路径。
- Verilator：
 - 周期级功能验证。

- Synopsys Design Compiler :
 - 面积;
 - 功耗;
 - 时序。

2. 工作负载

至少包含:

- 异构专家规模;
- 均匀专家规模;
- 均衡路由;
- 轻度倾斜路由;
- 高度倾斜路由;
- top-k = 1 / 2;
- 专家数 = 4 / 8 / 16;
- 不同 Token 数;
- 1-GPU / 2-GPU EP。

实验不能只依赖单一固定的 MoDSE Token 分布。

3. 硬件配置

列出:

- Systolic Array 大小;
- Dataflow;
- SRAM 总容量;
- Bank 数;
- 单 Bank 带宽;
- Bank 端口数;
- Request Buffer 深度;
- BankChannel 数;
- 默认 Prefetch Window;
- 默认 Chunk Size;
- 默认 GPU 数;
- 每 GPU 专家数。

4. Baseline

Baseline	Bank 分配	预取策略
Static-NoPF	静态	无
Static-NaivePF	静态	固定窗口、不感知 Bank

Baseline	Bank 分配	预取策略
Dynamic-NoPF	动态	无
Dynamic-NaivePF	动态	预取但不感知目标 Bank 压力
MemDomain	动态	Bank 感知 Chunk 预取
Oracle (可选)	使用未来信息	理论上界

Dynamic-NaivePF 必须保留，用于区分动态映射收益与真正的 Bank 感知协同收益。

5. 指标

性能指标

- Total Cycles;
- Speedup;
- Compute Cycles;
- Memory Stall Cycles;
- Stall Ratio;
- Throughput。

Bank 行为指标

- Bank Conflict Count / Rate;
- Bank Utilization;
- Bank Imbalance;
- Hotspot Bank Ratio;
- Idle Bank Ratio;
- Queue Length;
- Effective Bank Parallelism。

预取指标

- Prefetch Coverage;
- Prefetch Accuracy;
- Timely Prefetch Ratio;
- Late Prefetch Ratio;
- Unused Prefetch Ratio;
- Prefetch-induced Conflict;
- Compute-Transfer Overlap;
- Prefetch Occupancy;
- Mapping Count。

系统与硬件指标

- DRAM Traffic;
- Inter-GPU Communication Stall;
- Area;
- Power;
- Timing;
- Mapping Table 与 Prefetch Controller 开销。

B. 整体性能与 Bank 行为

研究问题

MemDomain 能带来多少端到端性能提升？性能收益是否确实来自 Bank 冲突缓解？

对比方案

- Static-NoPF;
- Static-NaivePF;
- Dynamic-NoPF;
- Dynamic-NaivePF;
- MemDomain。

展示结果

- 归一化 Total Cycles;
- Speedup;
- Memory Stall Cycles;
- Bank Conflict Rate;
- Hotspot / Idle Bank Ratio;
- Cycle Breakdown。

建议图

图 4：整体性能与 Bank 行为

- (a) 不同工作负载下的归一化执行周期;
 - (b) Memory Stall Breakdown;
 - (c) Bank Conflict / Hotspot / Idle 对比。
-

C. 组件消融实验

目标

量化每个机制的独立收益和协同收益。

建议顺序：

1. Static Baseline;
2. ◦ Unified Memory Domain;
3. ◦ Dynamic Bank Virtualization;
4. ◦ Naive Prefetch;
5. ◦ Bank-Aware Placement;
6. Full MemDomain。

需要回答

- 统一资源域本身是否带来收益？
- 动态映射贡献了多少收益？
- Naive Prefetch 是否引入新的冲突？
- Bank-Aware Placement 还能带来多少额外收益？
- Virtualization 与 Prefetch 是否互补？

D. Window 与 Chunk 敏感性

Sweep

- ($W = 0, 1, 2, 4, 8$);
- Chunk Size = 1、2、4、8 Tiles。

指标

- Total Cycles;
- Memory Stall Cycles;
- Prefetch-induced Conflict;
- Timely / Late / Unused Prefetch;
- SRAM Occupancy;
- Mapping Count;
- Compute-Transfer Overlap。

分析目标

- Window 太小 → 预取过晚;
- Window 太大 → 提前占用并干扰计算;
- Chunk 太小 → 控制和映射开销高;
- Chunk 太大 → 灵活性低、占用时间长。

为节省版面，建议使用二维 Heatmap。

E. 路由与专家配置敏感性

Sweep

- top-k = 1 / 2;
- 专家数 = 4 / 8 / 16;
- Token 数;
- 均衡 / 轻度倾斜 / 高度倾斜路由;
- 均匀 / 异构专家规模;
- 每卡专家数;
- 1-GPU / 2-GPU EP。

指标

- Speedup;
- Bank Conflict Reduction;
- Bank Imbalance;
- Memory Stall Reduction;
- Prefetch Timeliness;
- Communication Stall 与 Local Memory Stall。

核心问题

MemDomain 是否只对某一个人构造的路由分布有效，还是能够适应不同 MoE 配置？

F. 硬件开销与综合结果

不要使用“DC”作为小节标题。

报告内容

- 工艺节点;
- 时钟周期;
- 面积;
- 动态功耗;
- 静态功耗;
- 关键路径;
- Frontend / Midend / Backend 分解;
- Mapping Table;
- Pressure Counter;
- Prefetch Queue;

- Chunk Metadata;
- 控制逻辑。

注意

本科毕设中的面积功耗结果只能作为旧版本参考。DATE 扩展架构加入以下模块后，需要重新综合：

- Prefetch Queue;
- Chunk Metadata;
- Pressure-aware Bank Selection;
- Multi-GPU / EP 相关状态。

建议表格

表 II：硬件开销

模块	面积	功耗	功能
Unified Path	[]	[]	统一访问
Mapping Table	[]	[]	虚拟化
Pressure Counter	[]	[]	Bank 感知选择
Prefetch Controller	[]	[]	Chunk 调度
Full MemDomain	[]	[]	完整设计

V. 相关工作

A. MoE 路由与负载均衡

讨论：

- Token Balancing;
- Expert Load Balancing;
- Memory-aware Routing;
- Heterogeneous Experts;
- Expert Placement。

定位：

这些工作改变路由或专家放置，而 MemDomain 保持路由语义不变，管理路由结果产生的片上访存流。

B. MoE 通信、卸载与预取

讨论：

- All-to-All 优化;
- EP 系统;

- CPU-GPU Expert Offloading;
- Expert Caching;
- Pre-gated MoE;
- STEP 及其他专家预测预取工作。

定位:

现有方法主要决定哪些专家应被加载、缓存或通信，而 MemDomain 关注路由完成后专家 Chunk 的片上 Bank 放置与调度。

C. 片上存储虚拟化与 Bank 管理

讨论:

- Scratchpad Management;
- Dynamic Bank Partitioning;
- Bank-aware Placement;
- Memory Virtualization;
- Prefetch Contention;
- Unified Accumulator / Scratchpad。

定位:

现有设计主要面向规则 DNN 工作负载或固定 Tensor 生命周期，而 MemDomain 面向 MoE 中由路由和专家异构引起的动态访存流。

VI. 总结

- MoE 的存储瓶颈不仅是参数量大，还来自路由不均衡和专家异构造成的时变 Bank 级访存流。
- 静态 Bank 划分和无 Bank 感知的预取无法持续匹配该访存流。
- MemDomain 通过统一片上资源域、动态虚拟 Bank 映射和 Bank 感知 Chunk 预取缓解冲突并隐藏数据搬运延迟。
- 该方案在获得性能收益的同时，仅引入可接受的硬件开销。

六页 DATE 论文的图表预算建议

图

1. **图 1:** MoE Pipeline + 性能瓶颈刻画
2. **图 2:** 根因分析
 - 动态 IA/Weight/OA 需求;
 - Static Bank Heatmap;
 - Naive Prefetch Interference。
3. **图 3:** MemDomain 架构 + 执行时间线
4. **图 4:** 整体性能 + Bank 行为

5. 图 5：消融 + Window/Chunk 敏感性

表

- 1. 表 I：Workload、硬件配置、Baseline、默认参数
- 2. 表 II：面积与功耗开销

建议篇幅分配

章节	建议篇幅
摘要	0.2 页
I. 引言	0.7 ~ 0.8 页
II. 动机与瓶颈分析	1.0 ~ 1.1 页
III. MemDomain 架构	1.3 ~ 1.5 页
IV. 实验评估	2.1 ~ 2.3 页
V. 相关工作	0.25 ~ 0.35 页
VI. 总结	0.1 ~ 0.15 页